PRÁCTICAS DE IPO

Yoel Arroyo
Iván González
Ana Isabel Molina
Carmen Lacave

Controles personalizados
GUI dinámicas

En esta sesión aprenderemos…

- Creación de controles personalizados y reutilizables
- Añadir y eliminar controles personalizados en tiempo de ejecución
- Crear ventanas personalizadas con fondo transparente
- Crear estilos reutilizables
  - Diccionarios de recursos
- Reproducir contenido multimedia

Ejemplo: GUI principal

- Crearemos un proyecto nuevo (**GUIDinamicas**)
- Cambiamos las propiedades la ventana principal
  - `Title`: “GUI Dinámicas - Controles Personalizados”
  - `Width`: 1000
  - `Height`: 650

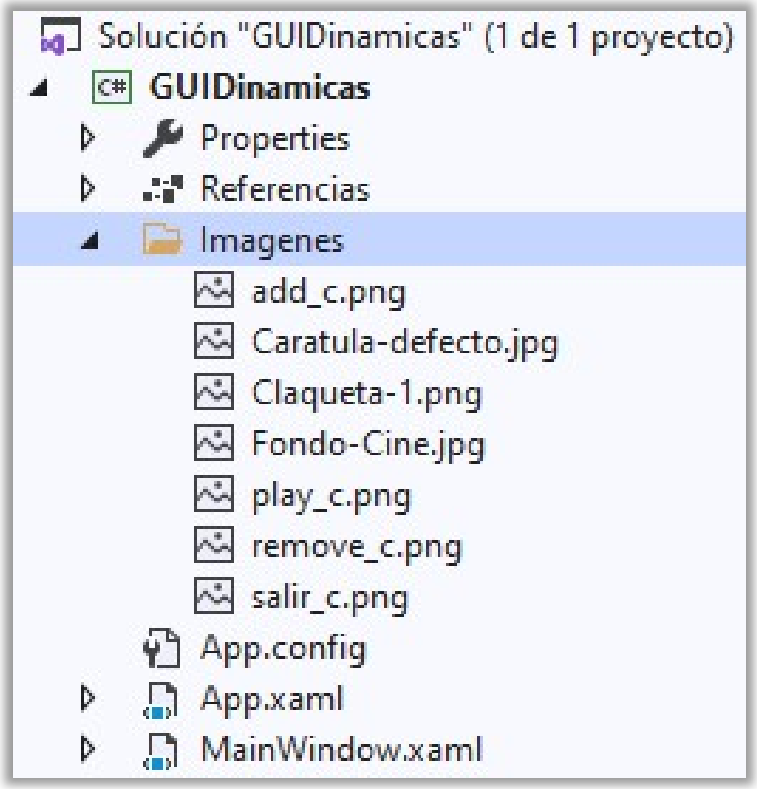
```
Title="GUI Dinámicas - Controles Personalizados" Height="650" Width="1000"
```

- El *layout* principal será de tipo `Grid` con 2 filas y 2 columnas
  - Primera fila
    - `Height="55"`
  - Primera columna
    - `Width="175*"`

```
<Grid>
    <Grid.RowDefinitions>
        <RowDefinition Height="55"/>
        <RowDefinition Height="576*"/>
    </Grid.RowDefinitions>
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="175*"/>
        <ColumnDefinition Width="195*"/>
    </Grid.ColumnDefinitions>
</Grid>
```

Añadir recursos al proyecto WPF

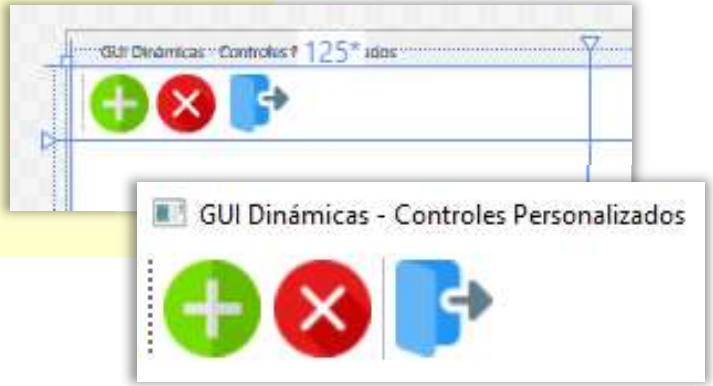
- Crearemos una carpeta para añadir los recursos (imágenes al proyecto)
  - Explorador de soluciones: *Botón derecho*>Agregar>Nueva Carpeta > “Imagenes“
  - *Botón derecho* sobre la nueva carpera> Agregar>Elemento existente>
  - Se seleccionan las imágenes a incluir en el proyecto



Diseño de la GUI principal

- En la primera fila se añadirá una Toolbar (tbBarraHerramientas)
  - Ocupará 2 columnas (Grid.ColumnSpan="2")
- Contendrá 3 botones con imágenes y un separador

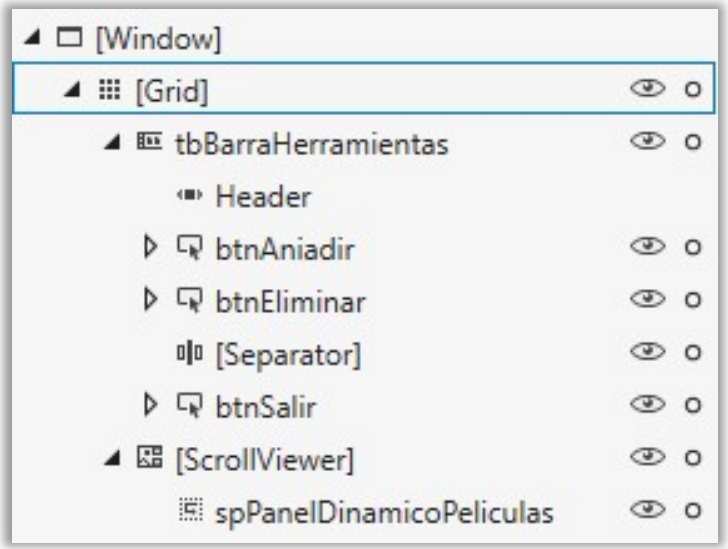
```
<ToolBar x:Name="tbBarraHerramientas"
         Grid.Row="0" Grid.ColumnSpan="2"
         Background="White">
    <Button x:Name="btnAniadir" ToolTip="Añadir película">
         <Image Source="Imagenes/add_c.png" />
    </Button>

    <Button x:Name="btnEliminar" ToolTip="Eliminar película">
         <Image Source="Imagenes/remove_c.png"/>
    </Button>
    <Separator></Separator>
    <Button x:Name="btnSalir" ToolTip="Salir">
         <Image Source="Imagenes/salir_c.png" />
    </Button>
</ToolBar>
```



Diseño de la GUI principal

- En la segunda fila y columna (`Grid.Row="1" Grid.Column="0"`) se añadirá un `WrapPanel` (`spPanelDinamicoPeliculas`) anidado en un `ScrollViewer`

```
<ScrollViewer Grid.Row="1" Grid.Column="0" Margin="5" >
    <WrapPanel x:Name="spPanelDinamicoPeliculas"/>
</ScrollViewer>
```

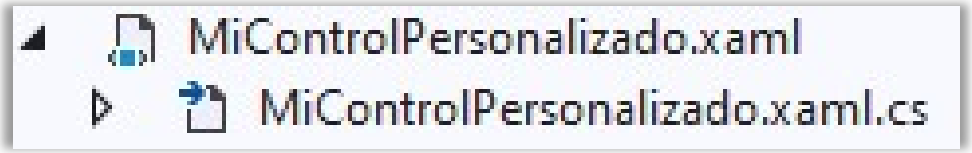


Controles personalizados

Creación de un Control Reutilizable

- Creamos el Control Personalizado:

  - Explorador de soluciones> *Botón derecho* sobre el proyecto>Agregar>**Control de usuario (WPF)**... **`MiControlPersonalizado.xaml`**
  - Se crean dos ficheros: **`MiControlPersonalizado.xaml`** y **`MiControlPersonalizado.xaml.cs`**

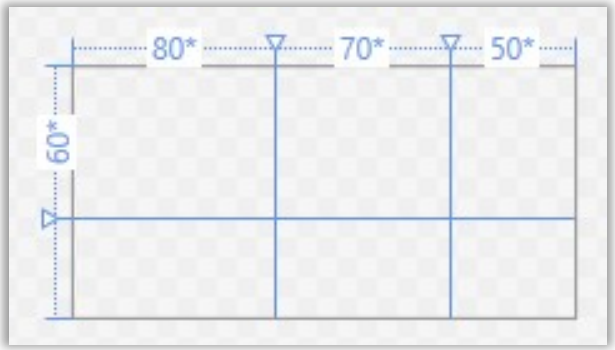


  - Contenido del fichero **`MiControlPersonalizado.xaml.cs`**

```
namespace GUIDinamicas
{
      /// <summary>
    /// Lógica de interacción para MiControlPersonalizado.xaml
    /// </summary>
    public partial class MiControlPersonalizado : UserControl
    {
        public MiControlPersonalizado()
        {
            InitializeComponent();
        }
    }
}
```

Diseño del Control Reutilizable

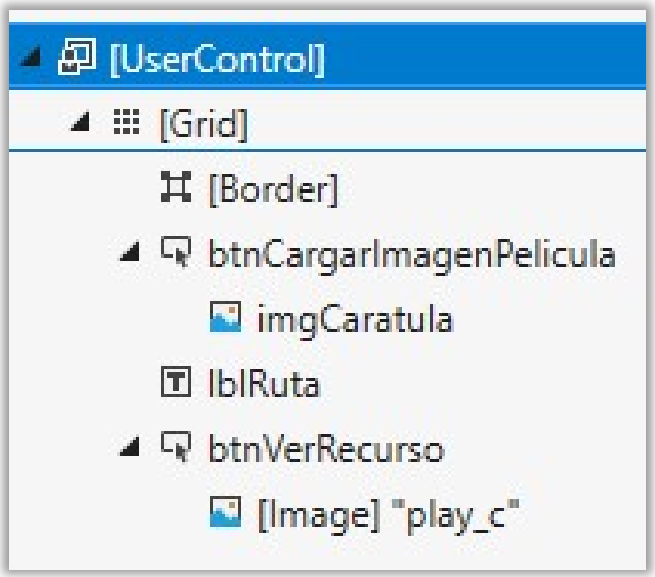
- Creamos el contenido del control personalizado reutilizable.
  - Tamaño (`Width="200" Height="100"`)
- En el Grid creamos 3 columnas y 2 filas
  - Añadimos un Border (ocupando las 3 columnas y las 2 filas)
    - `Grid.ColumnSpan="3" Grid.RowSpan="2"`
    - Modificamos las propiedades de color, fondo, grosor y radio de la esquina:

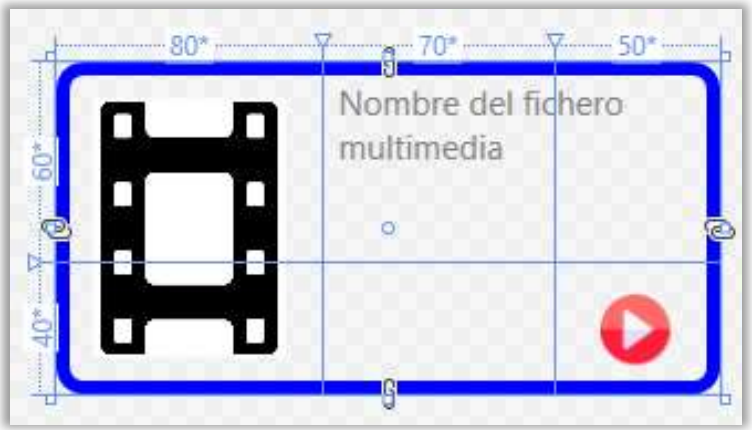



```
<Border Grid.ColumnSpan="3" Grid.RowSpan="2"
     BorderBrush="Blue"
     Background="Transparent"
     BorderThickness="4"
     CornerRadius="6" />
```

Diseño del Control Reutilizable

- Contenido del fichero **`MiControlPersonalizado.xaml`**

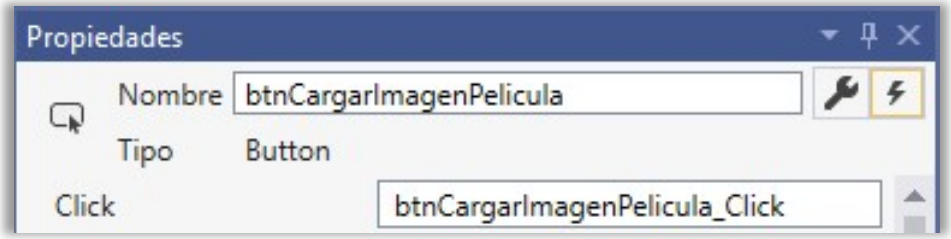





```xml
<Button x:Name="btnCargarImagenPelicula"
      Grid.Row="0" Grid.RowSpan="2"
      Width="60" Height="80"
      BorderBrush="Transparent" Background="White"
      ToolTip="Cambiar imagen">
            <Image x:Name="imgCaratula"
                  Source="Imagenes/Caratula-defecto.jpg"
                  Stretch="Fill" />
</Button>

<TextBlock x:Name="lblRuta"
      Text="Nombre del fichero multimedia"
      Grid.Column="1" Grid.ColumnSpan="2"
      Foreground="Gray"
      FontSize="10"
      TextWrapping="Wrap"
      Margin="5"/>

<Button x:Name="btnVerRecurso"
      Grid.Row="1" Grid.Column="2"
      Height="25" Width="25"
      BorderBrush="Transparent“ Background="Transparent"
      ToolTip="Cargar recurso multimedia">
            <Image Source="Imagenes/play_c.png"
                  Stretch="Fill"  />
</Button>
```

Diseño del Control Reutilizable

- Añadimos un evento `Click` al botón `btnCargarImagenPelicula`



- Incluimos el código del oyente en el fichero .cs
- En caso de error en `OpenFileDialog` hacer las importaciones correspondientes

```
private void btnCargarImagenPelicula_Click(object sender, RoutedEventArgs e)
{
     var abrirDialog = new OpenFileDialog();
     abrirDialog.Filter = "Images|*.jpg;*.gif;*.bmp;*.png";
     if (abrirDialog.ShowDialog() == true)
     {
          try
           {
               var bitmap = new BitmapImage(new Uri(abrirDialog.FileName, UriKind.Absolute));
               imgCaratula.Source = bitmap;
          }
          catch (Exception ex)
           {
               MessageBox.Show("Error al cargar la imagen " + ex.Message);
          }
     }
}
```

Oyentes para añadir y eliminar el control personalizado en tiempo de ejecución

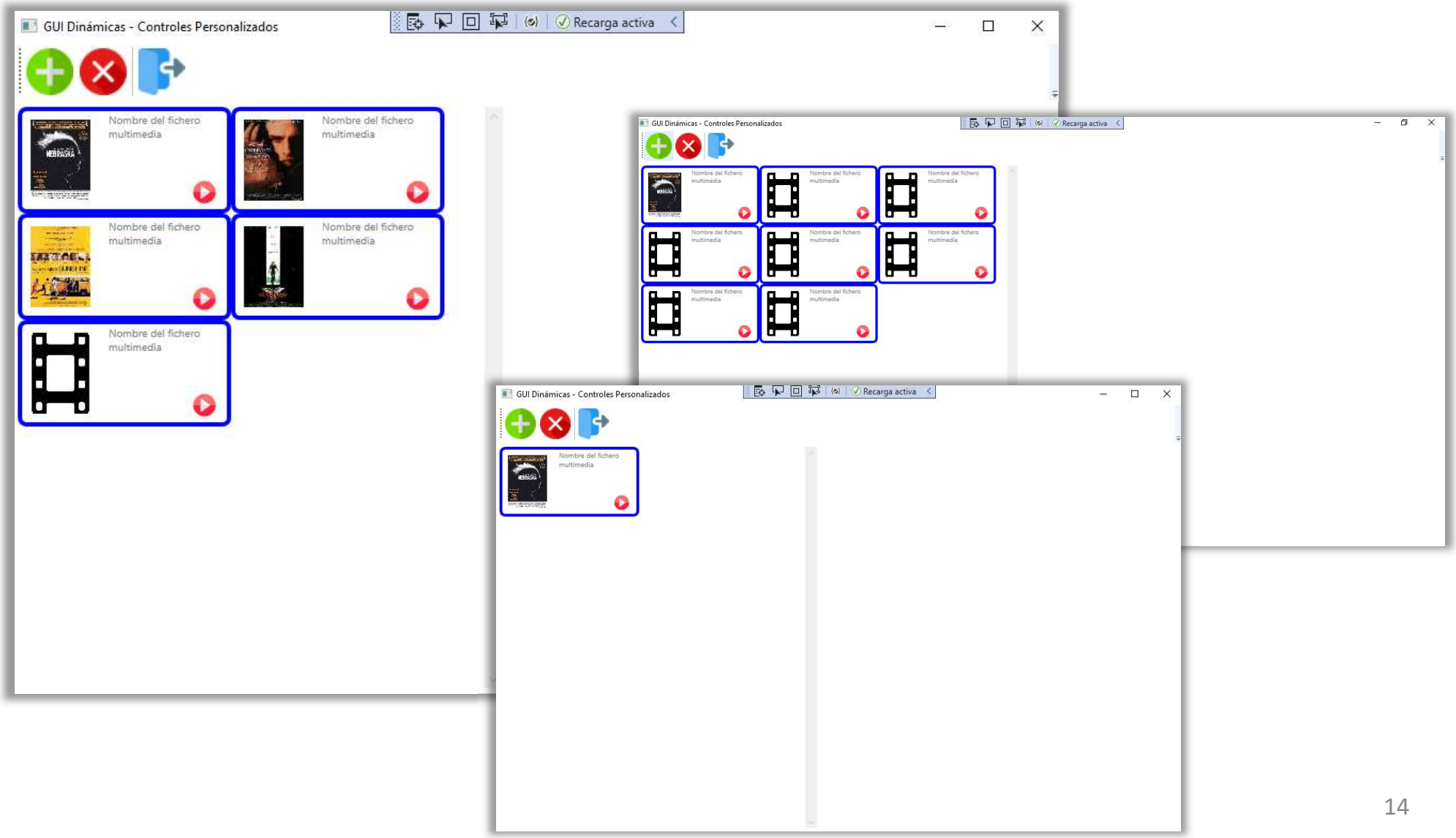
- Añadimos oyentes a los dos botones de la `Toolbar` (`btnAniadir` y `btnEliminar`) en el fichero **`MainWindow.xaml.cs`**.

```
private void btnAniadir_Click(object sender, RoutedEventArgs e)
{
    spPanelDinamicoPeliculas.Children.Add(new MiControlPersonalizado());
}
```

```
private void btnEliminar_Click(object sender, RoutedEventArgs e)
{
    if (spPanelDinamicoPeliculas.Children.Count >= 1)
    {
        spPanelDinamicoPeliculas.Children.RemoveAt(spPanelDinamicoPeliculas.Children.Count - 1);
    }
}
```

Oyentes para añadir y eliminar controles en tiempo de ejecución

- Resultado de ejecutar la aplicación:

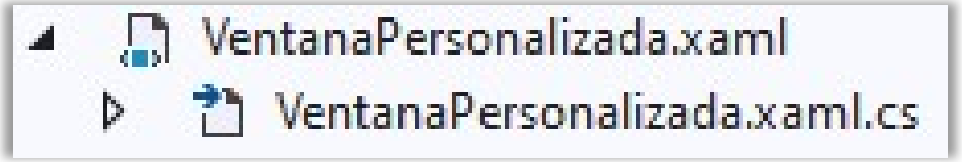


Creación de una ventana personalizada

Creación de una ventana personalizada

- Creamos una nueva ventana:
  - Explorador de soluciones> *Botón derecho* sobre el proyecto>Agregar>**Ventana (WPF)**… **`VentanaPersonalizada.xaml`**

  - Se crean dos ficheros: **`VentanaPersonalizada.xaml`** y **`VentanaPersonalizada.xaml.cs`**



  - Contenido del fichero **`VentanaPersonalizada.xaml.cs`**

```
namespace GUIDinamicas
{
    /// <summary>
    /// Lógica de interacción para VentanaPersonalizada.xaml
    /// </summary>
    public partial class VentanaPersonalizada : Window
    {
        public VentanaPersonalizada()
        {
            InitializeComponent();
        }
    }
}
```

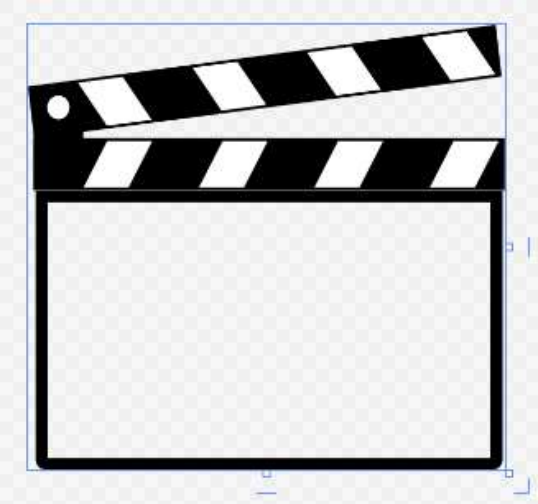
Diseño de la ventana personalizada

- Propiedades de la ventana personalizada:

```
Title="VentanaPersonalizada"
Height="270" Width="290"
AllowsTransparency="True"
WindowStyle="None"
ResizeMode="NoResize"
WindowStartupLocation="CenterScreen"
```

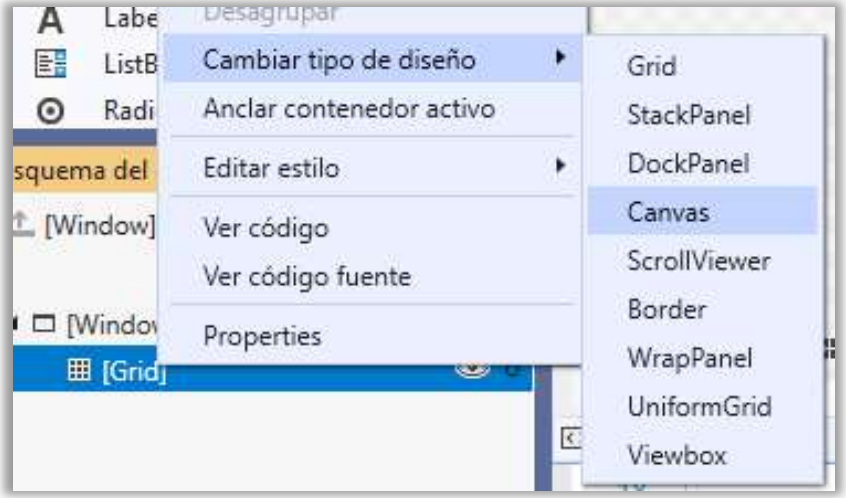
- Creamos, mediante una imagen con fondo transparente el aspecto de la ventana

```
<Window.Background>
     <ImageBrush ImageSource="Imagenes/Claqueta.png"/>
</Window.Background>
```



Diseño de la ventana personalizada

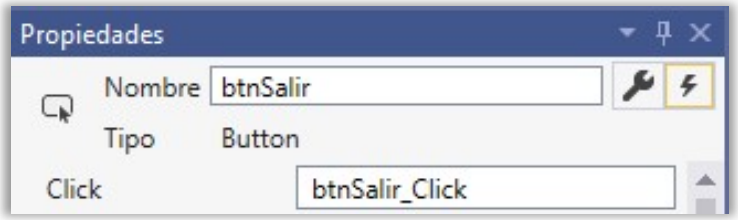
- Diseñamos el contenido de la ventana personalizada:
  - Sustituimos el layout creado por defecto en la ventana (`Grid`) por un `Canvas`



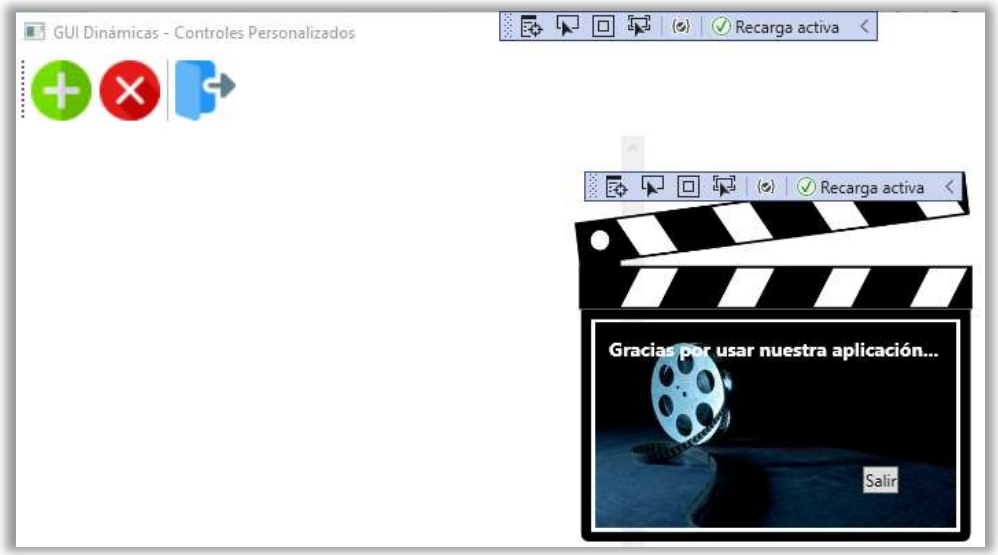

```
<Canvas>
    <Image Canvas.Left="15" Canvas.Top="110"
        Source="Imagenes/Fondo-Cine.jpg“
        Width="263" Height="150"
        Stretch="Fill"/>

    <TextBlock Canvas.Left=“25" Canvas.Top="120"
        Text="Gracias por usar nuestra aplicación..."
        FontSize="14" FontWeight="Bold"
        Foreground="White"/>

    <Button x:Name="btnSalir" Content="Salir"
        Canvas.Left="210" Canvas.Top="215"/>
</Canvas>
```

Lanzar la ventana personalizada desde la ventana principal

- Añadimos un oyente de evento de tipo `Click` al botón `btnSalir` (`MainWindow.xaml.cs`)

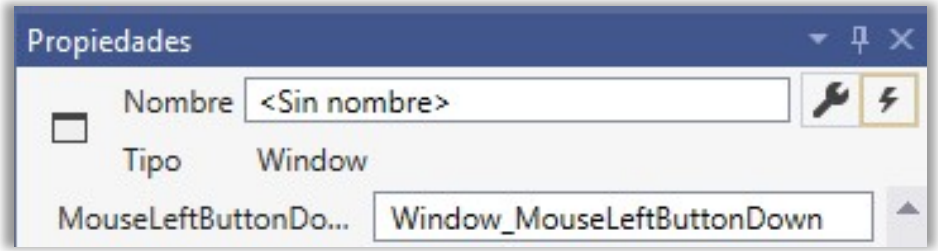



```
private void btnSalir_Click(object sender, RoutedEventArgs e)
{
    VentanaPersonalizada ventanaDespedida = new VentanaPersonalizada();
    ventanaDespedida.Show();
}
```



Lanzar la ventana personalizada desde la ventana principal

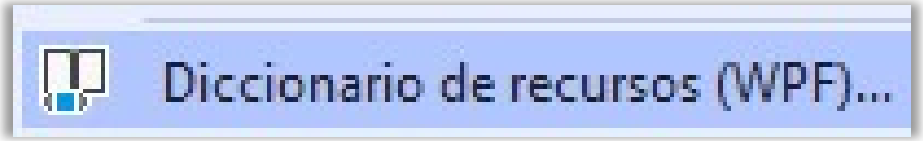
- La ventana no puede ser arrastrada por la pantalla:
  - Añadimos un oyente de tipo `MouseLeftButtonDown` que llame a `DragMove()`.




```
private void Window_MouseLeftButtonDown(object sender, MouseButtonEventArgs e)
{
    DragMove();
}
```

Creación de un diccionario de recursos

- Mejoraremos el aspecto del botón salir creando un estilo personalizado.
- Vamos a crear un **estilo reutilizable**, que puedo aplicar a distintos botones en la aplicación (se añade al **diccionario de recursos de la aplicación)**:
  - Creamos un fichero de estilos:
    - Explorador de soluciones> *Botón derecho* sobre el proyecto>Agregar>**Diccionario de recursos (WPF)**… `Estilos.xaml`



ResourceDictionary | ResourceDictionary

```
1  <ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
2                      xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
3
4  </ResourceDictionary>
```

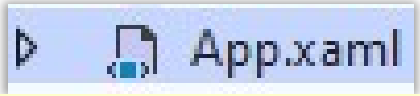
Creación de un estilo reutilizable

- Definimos las propiedades que definen el nuevo estilo del botón (en **`Estilos.xaml`**):

```
<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
                    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Style x:Key="EstiloBoton" TargetType="Button">
        <Setter Property="Background" Value="#CCCCFF"/>
        <Setter Property="Width" Value="80"/>
        <Setter Property="Height" Value="35"/>
        <Setter Property="Margin" Value="5"/>
        <Setter Property="FontSize" Value="16"/>
        <Setter Property="FontWeight" Value="Bold"/>
    </Style>
</ResourceDictionary>
```

Creación de un estilo reutilizable

- Declaramos en la aplicación que se hará uso de un diccionario de recursos.
  - Editar el fichero **App.xaml**




```
<Application x:Class="GUIDinamicas.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:local="clr-namespace:GUIDinamicas"
             StartupUri="MainWindow.xaml">

    <Application.Resources>
        <ResourceDictionary x:Name="RecursosEstilos" Source="Estilos.xaml" />
    </Application.Resources>

</Application>
```

  - Ya podemos hacer uso del estilo personalizado (**VentanaPersonalizada.xaml**)

```
<Button x:Name="btnSalir" Content="Salir"
        Canvas.Left="180" Canvas.Top="210"
        Style="{StaticResource EstiloBoton}"/>
```

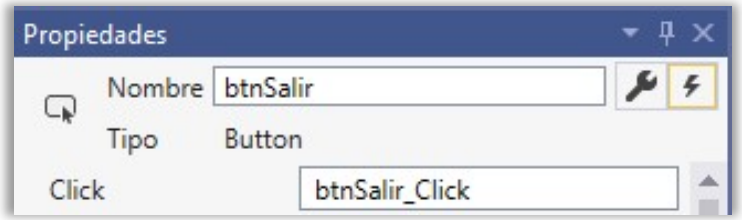
Creación de un estilo reutilizable

- Podemos mejorar aún más el aspecto del botón.
- Basta con editar el fichero **`Estilos.xaml`** añadiendo nuevas características (por ejemplo, una `Template` para incluir un borde redondeado):

```
<ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
                    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Style x:Key="EstiloBoton" TargetType="Button">
        <Setter Property="Background" Value="#CCCCFF"/>

        …
        <Setter Property="FontWeight" Value="Bold"/>

        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="Button">
                    <Border CornerRadius="15"
                            BorderThickness="1"
                            Background="{TemplateBinding Background}">
                        <ContentPresenter HorizontalAlignment="Center"
                                          VerticalAlignment="Center"/>
                    </Border>
                </ControlTemplate>
            </Setter.Value>
        </Setter>

    </Style>
</ResourceDictionary>
```

Oyente del botón salir de la ventana personalizada

- Añadimos un oyente de evento de tipo `Click` al botón `btnSalir` (`VentanaPersonalizada.xaml.cs`)




```
private void btnSalir_Click(object sender, RoutedEventArgs e)
{
    Application.Current.Shutdown();
}
```

Añadimos un botón Cancelar

- Para ver la diferencia entre cerrar la aplicación y cerrar la segunda ventana, podemos crear un botón Cancelar que haga que se pueda volver a la ventana principal.
  - Creamos el botón (`btnCancelar`)
  - Hacemos corta-pega del `btnSalir`.
  - Compartirá las mismas características pero distinta posición en el `Canvas`

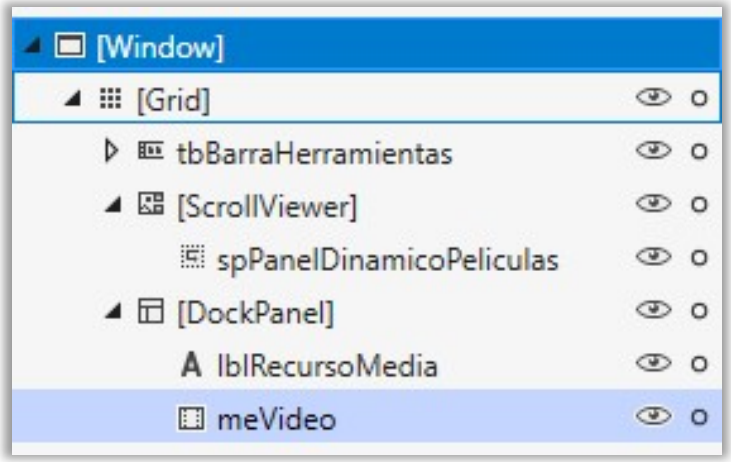
```
<Button x:Name="btnSalir" Content="Salir"
        Canvas.Left="180" Canvas.Top="210"
        Style="{StaticResource EstiloBoton}" Click="btnSalir_Click"/>
<Button x:Name="btnCancelar" Content="Cancelar"
        Canvas.Left="91" Canvas.Top="210"
        Style="{StaticResource EstiloBoton}" Click="btnCancelar_Click"/>
```

```
private void btnCancelar_Click(object sender, RoutedEventArgs e)
{
    Close();
}
```

Reproducir contenidos multimedia

Diseño de la GUI principal

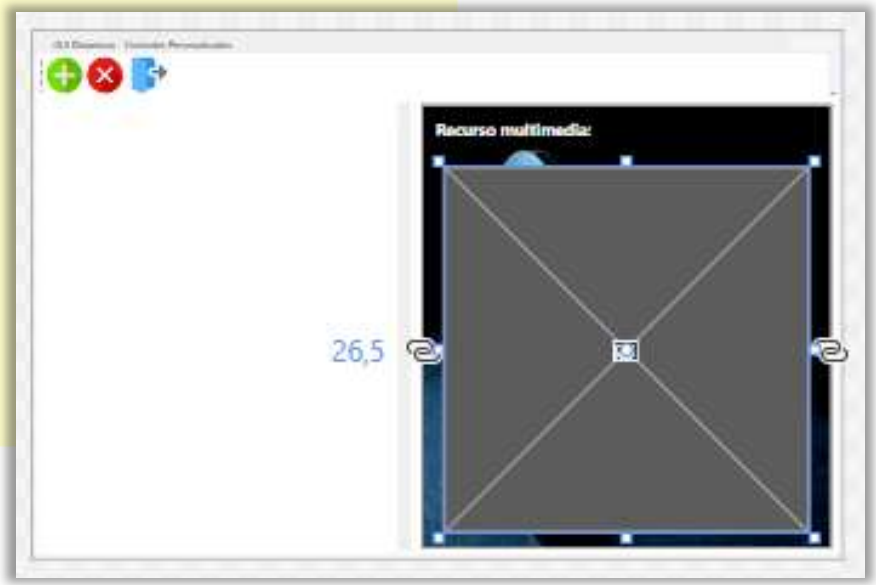
- En la segunda columna de la GUI principal, ocupando las dos filas (`Grid.Row="1" Grid.Column="1"`) se añadirá un elemento para reproducir contenido multimedia (`MediaElement`) para, por ejemplo, reproducir el trailer de una película o el audio de su banda sonora.
  - Añadimos un layout de tipo `DockPanel`
  - Cambiaremos el `background` del `DockPanel` para que incluya una imagen de fondo
    - Imagen: Fondo-Cine.jpg
  - En el `Top` añadiremos una `Label` (`lblRecursoMultimedia`)
    - Tamaño de letra: 20; negrita; letra en blando y fondo transparente.
  - En el `Botton` un elemento de tipo `MediaElement` (`meVideo`)
    - Tamaño: 450x450



Diseño de la GUI principal

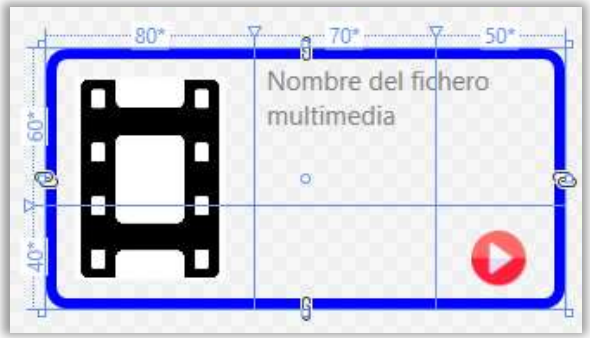
- La definición de esta parte de la GUI queda:

```
<DockPanel Grid.Column="1" Grid.Row="1"
     Margin="10">
     <DockPanel.Background>
          <ImageBrush ImageSource="Imagenes/Fondo-Cine.jpg"/>
     </DockPanel.Background>

     <Label x:Name="lblRecursoMedia" DockPanel.Dock="Top"
          Content="Recurso multimedia:"
          Margin="10"
          FontSize="20" FontWeight="Bold"
          Foreground="White"
          Background="Transparent"/>

     <MediaElement x:Name="meVideo"
          DockPanel.Dock="Bottom"
          Margin="10"/>

</DockPanel>
```



Carga de recursos multimedia

- Haremos que el botón `btnVerRecurso` del control personalizado permita cargar contenido multimedia en el `MediaElement` (`meVideo`)
- Añadimos un oyente de `Click` a dicho botón (fichero **`MiControlPersonalizado.xaml`**)




```
<Button x:Name="btnVerRecurso"
      Grid.Row="1" Grid.Column="2"
      Height="25" Width="25"
      BorderBrush="Transparent“ Background="Transparent"
      ToolTip="Cargar recurso multimedia“
      Click="btnVerRecurso_Click">
            <Image Source="Imagenes/play_c.png"
                   Stretch="Fill" />
</Button>
```

Carga de recursos multimedia

- Para poder acceder al reproductor del recurso multimedia (`meVideo`, que está en la GUI principal) se debería pasar una referencia al mismo a `MiControlPersonalizado` en su constructor.
  - El control personalizado almacenará dicha referencia en un atributo de clase.
- Modificaciones en el fichero `MiControlPersonalizado.xaml.cs`:

```
namespace GUIDinamicas
{
    /// <summary>
    /// Lógica de interacción para MiControlPersonalizado.xaml
    /// </summary>
    public partial class MiControlPersonalizado : UserControl
    {
        private MediaElement MeVideoExterno;

        public MiControlPersonalizado(MediaElement meVideoExterno)
        {
            MeVideoExterno = meVideoExterno;

            InitializeComponent();
        }

          …
}
```

Carga de recursos multimedia

- En el código en que se crean las instancias de `MiControlPersonalizado` (oyente `btnAniadir_Click`) en el fichero **`MainWindow.xaml.cs`** se deben cambiar la instrucción:

```
spPanelDinamicoPeliculas.Children.Add(new MiControlPersonalizado());
```

- por:

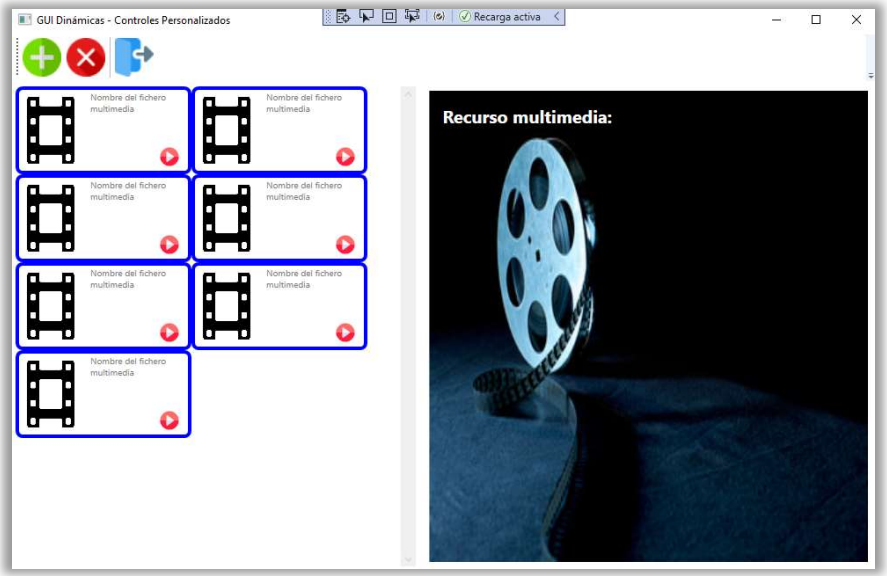
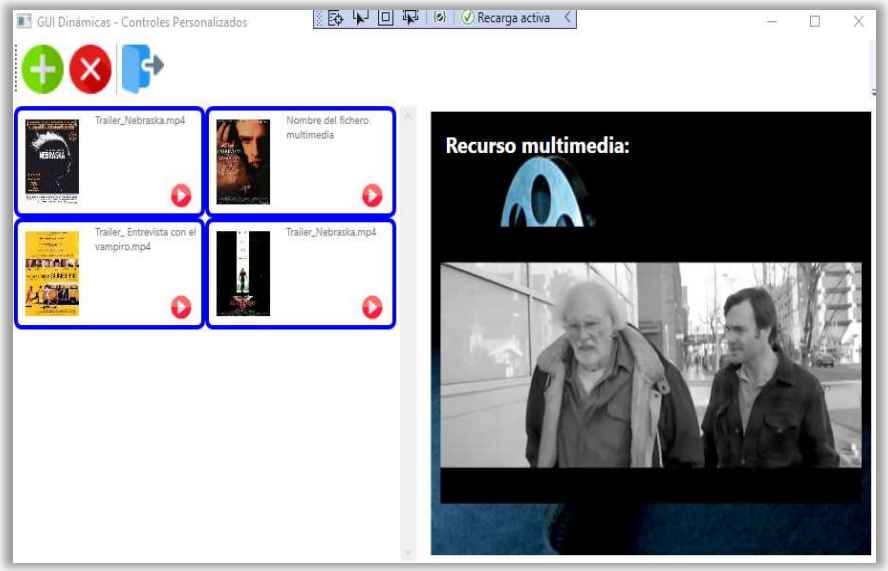
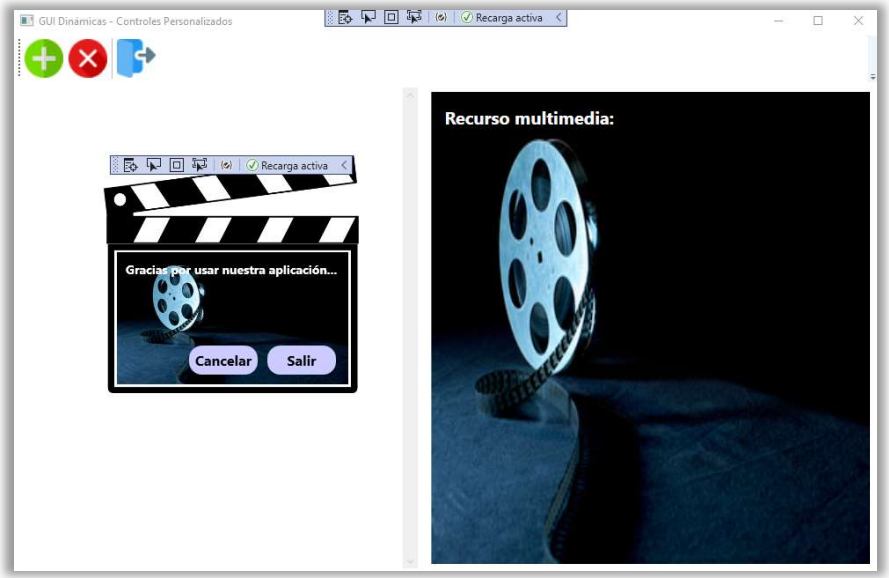
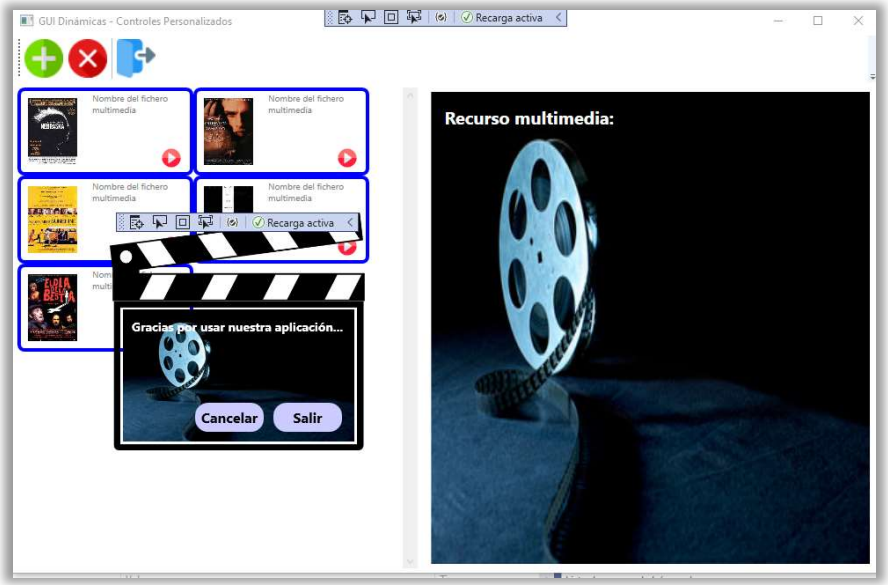
```
spPanelDinamicoPeliculas.Children.Add(new MiControlPersonalizado(meVideo));
```

Carga de recursos multimedia

- Contenido del oyente `btnVerRecursoClick`:

```
private void btnVerRecurso_Click(object sender, RoutedEventArgs e)
{
     var abrirDialog = new OpenFileDialog();
     if (abrirDialog.ShowDialog() == true)
     {
          try
          {
               MeVideoExterno.Source = new Uri(abrirDialog.FileName, UriKind.Absolute);
               lblRuta.Text = Path.GetFileName(abrirDialog.FileName);
          }
          catch (Exception ex)
          {
               MessageBox.Show("Error al cargar el fichero de video " + ex.Message);
          }
     }
}
```

```
using Path = System.IO.Path;
```

Resultado final de la aplicación

- Ejecutar la aplicación para comprobar el resultado final: